COMPRENDRE LE PROJET

Key Crypto V1 simplifiée

Guide complet pour les entretiens techniques et le CV

Python - API - pandas - SQL - statistiques - visualisation - tests - GitHub



Collecte manuelle > contrôle qualité > upsert > analyse locale

Le dashboard lit SQLite. Les interactions ne déclenchent pas l'API.

Un fil conducteur : partir d'une réponse API, établir une observation fiable, puis expliquer exactement ce qu'un indicateur permet de conclure.

27 chapitres, questions-réponses, exercices corrigés et annexe des sources réellement livrées.

État vérifié : 18 cas de test locaux réussis. Démonstration sans clé API. Les capacités prévues et les limites sont distinguées des résultats démontrés.

Sommaire

Sommaire 2

1. Ce que tu construis et ce que tu dois apprendre 4

2. Le besoin et la valeur du projet 5

3. Architecture : suivre une ligne de bout en bout 6

4. Pourquoi ces technologies et leurs alternatives 7

5. Installation et première exécution 8

6. Configuration centralisée 9

7. Client REST et erreurs compréhensibles 10

8. Nettoyage et alignement des données 11

9. Qualité : ce qui est signalé et ce qui ne l'est pas 12

10. SQLite, clé primaire et upsert 13

11. Orchestration, traces et collecte partielle 14

12. Calcul 1 : variation quotidienne 15

13. Calcul 2 : moyenne mobile 7 jours 16

14. Calcul 3 : volatilité 30 jours 17

15. Calcul 4 : comparaison en base 100 18

16. Dashboard : une page, des choix simples 19

17. Démonstration et tests : interpréter les preuves 20

18. GitHub, CI et protection des secrets 21

19. Limites à connaître et difficultés à expliquer 22

20. Plan d'apprentissage sur quatre semaines 23

21. Valoriser le projet sur le CV selon ton avancement 24

22. Adapter le CV au poste visé 25

23. Présenter le projet en entretien et expliquer l'assistance IA 26

24. Questions-réponses techniques 27

25. Exercices corrigés pour vérifier ton niveau 29

26. Démonstration de cinq minutes et liste de contrôle 30

27. Glossaire et références 31

Annexe A. Fiches des modules et lecture du code 32

A1. pyproject.toml 33

A2. .env.example 34

A3. .gitignore 35

A4. src/crypto_etl/config.py 36

A5. src/crypto_etl/client.py 37

A6. src/crypto_etl/transform.py 38

A7. src/crypto_etl/quality.py 39

A8. src/crypto_etl/storage.py 40

A9. src/crypto_etl/pipeline.py 42

A10. src/crypto_etl/indicators.py 43

A11. src/crypto_etl/demo.py 44

A12. src/crypto_etl/cli.py 45

A13. dashboard/app.py 46

A14. tests/test_core.py 48

A15. tests/test_dashboard.py 51

A16. .github/workflows/ci.yml 52

A17. .streamlit/config.toml 53

Annexe B. Commandes de révision 54

1. Ce que tu construis et ce que tu dois apprendre

Key Crypto V1 est un projet étudiant de traitement de données. Il récupère des observations de Bitcoin, Ethereum et Solana, les nettoie, les conserve dans SQLite et les affiche dans une page Streamlit. Son objectif est de rendre une chaîne de données compréhensible, vérifiable et présentable en entretien.

Le périmètre par défaut est de 90 jours, une observation par jour UTC terminé, en USD. Une collecte manuelle fait trois requêtes, une par actif. Quatre indicateurs sont retenus : variation quotidienne, moyenne mobile 7 jours, volatilité 30 jours et prix normalisé en base 100. L'utilisateur filtre les actifs et la période, puis peut exporter un CSV.

Le code existe déjà et a été construit avec assistance IA. Ton travail personnel consiste à comprendre chaque étape, la réexécuter et savoir faire une petite modification. Présenter le projet comme maîtrisé demande davantage que télécharger le dépôt : tu dois relier une règle métier à une fonction et à un test.

La dernière validation locale de la V1 comprend 18 cas de test réussis, Ruff et son contrôle de formatage. Une démonstration neuve contient 270 observations synthétiques. Aucun volume réel de 50 000 observations n'est démontré par cette version. La collecte authentifiée reste à valider avec une vraie clé CoinGecko. Les résultats de CI distants sont à consulter dans l'onglet Actions ; ce guide ne présume pas leur état actuel.

Pour lire efficacement : commence par le besoin, le schéma du flux et les quatre calculs. Étudie ensuite les modules avec l'annexe de code. Termine par les exercices, les questions d'entretien et les formulations CV correspondant à ton niveau réel d'avancement.

2. Le besoin et la valeur du projet

Le besoin est de comparer des actifs dont les prix sont très différents tout en conservant une base locale propre. Un prix Bitcoin et un prix Solana ne se comparent pas directement en valeur absolue : la base 100 permet d'observer leur évolution relative depuis un point de départ.

La valeur technique ne vient pas du nombre de graphiques. Elle vient de trois garanties : relancer une collecte sans dupliquer les dates, ne pas inventer de prix manquant et conserver des données valides lorsqu'une source échoue. Ce sont des problèmes concrets rencontrés dans beaucoup de traitements de données, pas seulement en finance.

Une application qui appelle l'API à chaque filtre dépend du réseau et consomme du quota. Dans cette V1, le pipeline actualise SQLite manuellement et le dashboard lit cette base. On peut donc consulter les données déjà disponibles sans refaire une collecte.

Le projet démontre Python, pandas, une API REST, les clés et transactions SQL, la visualisation et des tests ciblés. Il ne démontre pas une stratégie d'investissement rentable, une architecture distribuée ou une application de production à grande échelle. Cette distinction rend sa présentation plus crédible.

3. Architecture : suivre une ligne de bout en bout



Collecte manuelle > contrôle qualité > upsert > analyse locale

Le dashboard lit SQLite. Les interactions ne déclenchent pas l'API.

Le chemin d'exécution est : commande CLI, configuration, client CoinGecko, archive JSON, transformation et qualité, chargement SQLite. Le dashboard suit un second chemin : SQLite, calendrier quotidien, calcul des indicateurs, filtres, graphiques et export.

Extract signifie extraire : recevoir les données de l'API. Transform signifie transformer : convertir les dates, aligner les séries et contrôler les valeurs. Load signifie charger : enregistrer le résultat validé en SQL. Le nom ETL correspond à cet ordre. L'archive brute est un support de diagnostic, pas la table utilisée par les graphiques.

Exemple : l'API fournit un timestamp et un prix de 100. La transformation détermine sa date UTC, vérifie que le prix est positif et fini, puis décide s'il s'agit du dernier point valide de ce jour. Le stockage insère ou met à jour la ligne identifiée par l'actif, la devise et la date. À la lecture, pandas calcule sa variation si le jour précédent est disponible.

Les indicateurs ne sont pas stockés en SQL. Ils sont recalculés au moment de l'affichage, ce qui évite de maintenir une seconde table dérivée. Si un prix historique est corrigé, les fenêtres concernées sont recalculées à la prochaine lecture.

Les fonctions sont séparées par responsabilité. config.py charge les paramètres ; client.py parle HTTP ; transform.py nettoie ; quality.py décrit les anomalies ; storage.py porte SQL ; pipeline.py orchestre ; cli.py expose les commandes ; demo.py génère la démonstration ; indicators.py calcule ; dashboard/app.py affiche.

4. Pourquoi ces technologies et leurs alternatives

Python permet de traiter les appels HTTP, les données et l'interface dans un même langage. Cela limite les outils à apprendre pour un premier projet. Il faut cependant éviter de conclure que Python serait toujours le meilleur choix : ici, il convient au faible volume local et à l'objectif pédagogique.

requests rend visibles l'URL, les paramètres, l'en-tête et le timeout. Un SDK fournisseur aurait pu masquer une partie du contrat HTTP. La V1 utilise un client synchrone et un seul appel par actif ; elle n'a ni concurrence réseau ni reprise automatique.

pandas sert aux tableaux, aux jointures sur timestamp, aux calendriers et aux fenêtres glissantes. NumPy reste une dépendance indirecte de pandas mais n'est pas importé dans le projet. La bibliothèque standard math suffit au contrôle des valeurs finies. À gros volume, le chargement en mémoire et les calculs répétés demanderaient des mesures de performance.

SQLite fournit une base SQL sans serveur à administrer. Une clé primaire et un upsert sont plus adaptés aux mises à jour fiables qu'une accumulation de CSV. PostgreSQL serait à envisager pour plusieurs services écrivant simultanément ou une application partagée. Il n'est pas nécessaire pour démontrer les fondamentaux sur trois séries quotidiennes.

Streamlit permet de produire une interface analytique en Python. Plotly ajoute survol et zoom. Matplotlib conviendrait à des figures statiques, mais n'est pas utilisé dans cette V1. Une application web dédiée offrirait davantage de personnalisation au prix d'un périmètre plus large.

pytest vérifie les comportements et les résultats attendus. Ruff contrôle certains défauts de code et son formatage. GitHub Actions relance les contrôles sur plusieurs environnements. Aucun de ces outils ne prouve à lui seul la qualité économique d'une analyse ou l'absence de tous les bugs.

5. Installation et première exécution

Depuis le dossier du dépôt, créer un environnement avec python -m venv .venv. Sous PowerShell, activer avec .venv\Scripts\Activate.ps1 ; sous Linux, utiliser source .venv/bin/activate. Installer ensuite avec python -m pip install -e ".[dev]". L'installation éditable relie le paquet au code local ; les dépendances de développement ajoutent pytest et Ruff.

Lancer crypto-etl demo, puis streamlit run dashboard/app.py. Le premier programme écrit la base synthétique ; le second démarre un serveur web local. Le dashboard commence en mode démonstration, avec une bannière visible. Ces prix sont fabriqués et ne représentent pas le marché.

Si l'activation PowerShell ne fonctionne pas, appeler directement .venv\Scripts\python.exe -m crypto_etl.cli demo et .venv\Scripts\python.exe -m streamlit run dashboard/app.py. Cela évite de modifier une politique globale uniquement pour activer un environnement.

Pour une collecte réelle, charger COINGECKO_API_KEY dans le terminal, puis lancer crypto-etl update. Sous PowerShell : $env:COINGECKO_API_KEY="votre_cle". Sous Linux : export COINGECKO_API_KEY='votre_cle'. Désactiver le mode synthétique dans l'interface pour lire market.db. Ne jamais copier une vraie clé dans un document, une capture ou un commit.

Les tests et la démonstration n'ont besoin d'aucune API extérieure. Le téléchargement initial des dépendances nécessite Internet. Le moteur de test Streamlit utilise néanmoins des mécanismes locaux asyncio ; un environnement très restreint peut bloquer ses sockets locales sans que cela indique un défaut du dashboard.

6. Configuration centralisée

Config est une dataclass figée. Elle contient la clé, la devise, la durée, le dossier de données, les actifs, le timeout et l'URL. frozen=True empêche la réaffectation ordinaire des champs après construction. api_key utilise repr=False pour éviter d'apparaître dans la représentation de l'objet ; ce n'est pas un chiffrement du secret.

load_config lit quatre variables : COINGECKO_API_KEY, CRYPTO_CURRENCY, CRYPTO_DAYS et CRYPTO_DATA_DIR. Les valeurs par défaut sont clé vide, usd, 90 et data. .env.example est une documentation, pas un chargeur automatique : un fichier .env n'est pas lu par ce programme.

La durée est validée entre 1 et 365 jours. La devise doit être alphabétique mais n'est pas comparée à la liste des devises acceptées par CoinGecko. Une valeur alphabétique inexistante peut donc passer la validation locale puis échouer côté API. Une durée non convertible en entier échoue pendant le chargement de configuration.

db_path choisit demo.db ou market.db. Les bases et archives de démonstration sont séparées des données réelles. Un chemin relatif dépend du dossier de lancement : toujours lancer depuis le dépôt ou définir un chemin absolu pour CRYPTO_DATA_DIR.

La liste des actifs et le timeout sont des champs du code, pas des variables d'environnement documentées dans cette version. Modifier la liste demande aussi de vérifier la démonstration, qui possède des valeurs de base seulement pour les trois actifs prévus.

7. Client REST et erreurs compréhensibles

Client.fetch construit GET /coins/{coin_id}/market_chart avec vs_currency, days et interval=daily. La clé est transmise dans x-cg-demo-api-key. Les observations attendues arrivent dans prices, market_caps et total_volumes sous forme de paires timestamp/valeur.

Une session HTTP est injectable : en usage normal, requests.Session réalise l'appel ; dans un test, Mock renvoie une réponse fabriquée. Le timeout de 30 secondes limite les attentes réseau selon les mécanismes de requests. Il ne s'agit pas d'une garantie de durée maximale absolue de toute la collecte.

La V1 fait une tentative par actif. Une erreur réseau produit un message générique. Une réponse autre que 200 produit une erreur contenant le statut HTTP. Un JSON illisible est refusé. Il n'existe pas de boucle de reprise, de backoff ni de traitement Retry-After automatique : en cas de 429, attendre puis relancer manuellement.

validate_payload vérifie la présence des trois listes et la longueur des paires. Cette validation est appelée par transform après archivage du JSON décodable. Un objet JSON mal structuré peut ainsi être conservé pour diagnostic puis rejeté. Un corps non décodable en JSON n'est pas archivé comme réponse JSON dans cette version.

Le pipeline ne recopie pas les textes d'exceptions externes dans son bilan : il garde le type de l'erreur. Cela réduit le risque de fuite, mais limite les détails de diagnostic. Pour apprendre, commencer par les codes HTTP : 400 indique une requête refusée, 401 une authentification refusée, 429 une limitation et 500 une erreur serveur. Les causes exactes restent dépendantes du fournisseur.

8. Nettoyage et alignement des données

transform reçoit un objet JSON, un actif, une devise, une durée et éventuellement une date now pour les tests. Elle détermine le jour courant UTC et la fenêtre des jours terminés. Le jour UTC en cours et les dates futures sont exclus, car ils ne constituent pas encore une observation quotidienne complète.

Les trois séries sont converties en DataFrames séparés. Le timestamp devient numérique ; les timestamps non convertibles sont retirés. Les doublons exacts de timestamp sont résolus en gardant la dernière occurrence de chaque série. Les tableaux sont ensuite alignés sur cette clé avec concat, pas sur la position des lignes.

Exemple : les prix existent pour le 1er et le 2 janvier, mais la capitalisation seulement pour le 2 janvier. La capitalisation doit être manquante au 1er janvier, et présente au 2. Associer les premiers éléments des deux listes ferait une erreur silencieuse. C'est pourquoi la jointure doit utiliser une clé explicite.

pd.to_datetime convertit les millisecondes avec utc=True et errors='coerce'. Après conversion en UTC, le fuseau est retiré et l'heure normalisée afin de conserver une date quotidienne. Le nom date_utc garde cette convention. Un timestamp non représentable devient une date manquante.

Chaque valeur devient numérique. math.isfinite exclut infinis et NaN. Un prix doit être strictement positif : zéro, négatif ou manquant entraîne un rejet. Volume et capitalisation peuvent être nuls ; s'ils sont invalides, leur valeur est absente sans rejeter un prix valide.

Les lignes valides dans la fenêtre sont triées par timestamp, puis la dernière observation valide de chaque jour est conservée. Il n'y a pas de remplissage d'une autre série depuis un timestamp voisin. Il n'y a pas d'interpolation des prix. Le résultat est trié par date et porte l'actif et la devise.

9. Qualité : ce qui est signalé et ce qui ne l'est pas

quality_report construit les dates attendues de la fenêtre et compare aux dates retenues. missing_days liste les jours absents. Le bilan contient aussi accepted, rejected, duplicate_timestamps, excluded_current_or_future, missing_market_cap, missing_volume et large_changes.

Une variation absolue supérieure à 50 % est signalée, sans suppression arbitraire. Une variation importante peut être réelle. Le bilan est conservé avec l'exécution dans pipeline_runs ; la page simplifiée n'affiche pas une vue détaillée de tous ces compteurs.

Les compteurs ne sont pas une partition parfaite de tous les éléments bruts. Les doublons sont comptés par série, les timestamps non numériques sont retirés avant rejected et les rejets sont évalués avant le filtre temporel. Connaître ces limites évite d'affirmer que le bilan explique exhaustivement chaque point reçu.

Un jour absent de la nouvelle réponse peut déjà exister dans SQLite grâce à une ancienne collecte. La qualité d'une collecte et la qualité de toute la base sont donc différentes. Les données anciennes restent jusqu'à l'arrivée d'une nouvelle valeur valide ; la V1 n'efface pas automatiquement une ligne parce qu'elle manque dans une nouvelle réponse.

Dans le dashboard, les trous du calendrier sélectionné sont signalés. La fraîcheur est contrôlée par actif ; une donnée antérieure à la veille UTC donne un avertissement. Ce contrôle ne certifie pas que les chiffres sont économiquement exacts, mais rend visible un retard de collecte.

10. SQLite, clé primaire et upsert

daily_market_data contient actif, devise, date UTC, timestamp source, prix, capitalisation, volume 24 h et date de collecte. La clé primaire (coin_id, vs_currency, date_utc) définit l'unicité métier. La date seule ne suffit pas puisque plusieurs actifs et devises partagent le même jour.

INSERT ... ON CONFLICT ... DO UPDATE permet d'insérer une date nouvelle ou d'actualiser une ligne existante. Si la même réponse est chargée deux fois, le nombre de dates ne double pas. Si un prix historique est révisé, il est remplacé par la nouvelle valeur valide.

L'idempotence porte sur l'état métier attendu : absence de doublons et conservation du prix fourni. collected_at est actualisé à chaque chargement, donc rejouer un lot n'est pas nécessairement une opération identique bit à bit sur toutes les métadonnées.

Les écritures sont liées avec des placeholders SQL. Cela évite de concaténer des valeurs dans une requête. Les contraintes CHECK contrôlent les valeurs positives ou non négatives. Python convertit ses valeurs manquantes en None pour stocker SQL NULL, ce qui n'est pas la même chose que zéro.

La transaction couvre un actif. Si son lot échoue, ses écritures sont annulées ; les actifs déjà validés restent disponibles. Le context manager utilise closing pour fermer explicitement la connexion et le contexte sqlite3 pour commit/rollback. SQLite est adapté à l'usage local, sans gestion avancée de plusieurs collectes simultanées.

La base conserve la version actuelle d'une ligne, pas toutes ses révisions. Les archives JSON peuvent aider à enquêter. REAL SQLite représente les nombres en virgule flottante : ce choix convient à cette analyse descriptive, pas à une comptabilité financière exigeant un calcul décimal exact.

11. Orchestration, traces et collecte partielle

run crée un identifiant UUID et enregistre running dans pipeline_runs. Pour chaque actif : obtenir le JSON, l'archiver, transformer, vérifier qu'il existe au moins une ligne valide, puis faire l'upsert. Une erreur est isolée pour permettre aux autres actifs de continuer.

Le statut final est success si tous réussissent, partial si certains réussissent, failed si aucun ne réussit. rows_processed compte toutes les lignes traitées avec succès, y compris les lignes actualisées. Ce n'est pas le nombre de nouvelles insertions.

Exemple : une première collecte réussit pour les trois actifs. À la suivante, Ethereum échoue tandis que Bitcoin et Solana réussissent. Leurs nouvelles données sont chargées ; celles d'Ethereum restent anciennes ; le statut devient partial et le dashboard avertit de l'incomplétude et éventuellement de la fraîcheur.

Les archives se trouvent dans raw ou raw-demo, avec un sous-dossier par identifiant d'exécution. Il n'existe pas de transaction unique entre le système de fichiers et SQLite : une archive peut exister alors qu'un chargement a échoué. Une interruption brutale peut laisser running ; aucune récupération automatique de ce statut n'est implémentée.

La CLI expose demo et update. La commande renvoie 0 pour un succès complet, 1 pour une collecte partielle ou échouée, et argparse peut renvoyer 2 pour un usage invalide ou une clé manquante. Un incident de stockage avant le traitement peut encore faire échouer la commande directement.

12. Calcul 1 : variation quotidienne

La formule est 100 × (P(t)/P(t-1) - 1). Pour 100 puis 105, le rendement est 5 %. Pour 105 puis 100, il vaut environ -4,7619 %, pas -5 %. Le prix de référence est différent dans les deux sens.

Dans le code : price.pct_change(fill_method=None) * 100. fill_method=None évite de reporter automatiquement le dernier prix connu à un jour manquant. Les résultats sont en pourcentage : la valeur 5 signifie 5 %, pas 0,05 %.

indicators groupe par actif et devise puis applique asfreq('D'). Un jour absent apparaît comme NaN. Sans calendrier explicite, la ligne précédente pourrait dater de plusieurs jours et le résultat serait présenté à tort comme quotidien. Au premier jour, le rendement est indisponible car aucun prix précédent n'est présent.

Les rendements simples se composent. Deux hausses de 10 % donnent 1,1 × 1,1 = 1,21, donc 21 % au total. Additionner naïvement les rendements donne une approximation, pas une performance exacte. La V1 n'ajoute pas de rendement logarithmique.

13. Calcul 2 : moyenne mobile 7 jours

La MM7 est la moyenne des sept derniers prix quotidiens, jour observé inclus. Les prix 100, 101, 102, 103, 104, 105, 106 donnent 721/7 = 103. Le code utilise rolling(7, min_periods=7).mean(). Les six premiers résultats restent manquants.

Une moyenne mobile lisse les fluctuations. Elle est retardée par construction et ne prédit pas le prochain prix. Le fait qu'un prix soit au-dessus de sa moyenne ne prouve pas une hausse future et ne déclenche aucun achat dans cette application.

Une fenêtre exige sept prix valides. Si l'un des jours manque, la moyenne reste indisponible tant que cette date se trouve dans la fenêtre. La V1 ne remplace pas le prix absent par une valeur interpolée ou le prix précédent.

La MM7 est calculée avant le filtre d'affichage. Si la sélection commence le 10 juin mais que les données depuis le 4 juin existent, la moyenne du 10 juin peut utiliser ces jours. Filtrer d'abord provoquerait artificiellement six valeurs manquantes au début de chaque sélection.

14. Calcul 3 : volatilité 30 jours

La volatilité retenue est l'écart-type échantillonnal de 30 rendements quotidiens exprimés en pourcentage. Dans le code : return_1d.rolling(30, min_periods=30).std(ddof=1). Il faut 31 prix consécutifs pour obtenir 30 rendements valides.

Calcul pédagogique sur trois rendements 1 %, 2 %, 3 % : moyenne 2 %, écarts -1, 0 et 1 point de pourcentage ; somme des carrés 2 ; division par n-1 = 2 ; racine de 1 = 1 point de pourcentage. Avec ddof=0, le résultat serait environ 0,8165. L'exemple explique l'écart-type ; l'application utilise une fenêtre de 30, pas de trois rendements.

La mesure n'est pas annualisée. Une conversion annuelle demanderait une convention et des hypothèses supplémentaires. Un résultat élevé indique de fortes fluctuations observées, sans indiquer à lui seul la direction des prix. Un résultat faible ne garantit pas l'absence de choc futur.

Un seul prix manquant peut invalider deux rendements consécutifs, puis plusieurs fenêtres de volatilité. C'est un comportement attendu, pas une raison pour remplacer les résultats indisponibles par zéro. La V1 ne calcule ni VaR ni risque de portefeuille.

15. Calcul 4 : comparaison en base 100

La formule est 100 × prix / premier prix disponible de la sélection. Avec 200 au départ et 220 à la fin, la base passe de 100 à 110. Cela correspond à une hausse de 10 %, même si un autre actif possède un niveau de prix très différent.

La base 100 est recalculée après le filtre de période avec select_period. Elle répond à une question liée à l'intervalle choisi : comment les actifs ont-ils évolué depuis leur premier prix disponible dans cette sélection ? Changer le début change la référence.

Une base finale 120 signifie +20 % par rapport au prix de référence, pas +120 %. Le graphique compare les trajectoires relatives, pas la capitalisation des actifs ni le rendement net d'un portefeuille tenant compte des frais.

Si une crypto ne possède pas de prix au début de la période, son premier prix disponible est utilisé. Deux actifs peuvent alors avoir des dates de référence différentes. Il faut vérifier ces dates avant de tirer une conclusion de comparaison ; une option d'alignement sur dates communes est une amélioration possible, non incluse.

16. Dashboard : une page, des choix simples

La page démarre en mode synthétique. Elle vérifie l'existence de la base puis l'existence de données pour la devise. Un état vide donne la commande d'initialisation plutôt qu'un tableau d'erreur. La dernière collecte et les données anciennes sont signalées.

La barre latérale permet de sélectionner les actifs et les dates. Le tableau montre le dernier prix disponible dans cette sélection, sa date, sa variation quotidienne et sa volatilité. Le graphique base 100 compare les actifs retenus. Un sélecteur choisit ensuite l'actif du graphique prix/MM7 et de la volatilité.

Le bouton export produit un CSV du tableau sélectionné, avec ses indicateurs et ses éventuelles lignes calendaires manquantes. Les fichiers portent un nom distinct selon le mode démo ou réel. La clé API n'est pas incluse dans les données exportées.

Streamlit réexécute le script lors des interactions. La V1 relit SQLite et recalcule sans cache. Pour ce volume réduit, cela garde le fonctionnement simple. Les filtres n'écrivent pas de prix et n'appellent jamais CoinGecko. Plotly fournit survol et zoom ; ce n'est pas une interface de trading.

Le volume et la capitalisation restent dans le stockage mais ne disposent pas de graphiques dédiés dans cette page. Garder les données sources disponibles n'oblige pas à élargir la présentation. Les messages et unités doivent toujours être lus avec la devise configurée et les dates UTC.

17. Démonstration et tests : interpréter les preuves

demo.py fabrique des séries déterministes selon la position dans la fenêtre, avec une progression et une sinusoïde. La fin du calendrier suit la veille UTC. Une base neuve contient 90 × 3 = 270 lignes ; une ancienne base peut en contenir davantage car le pipeline ne supprime pas les anciennes dates.

La suite actuelle exécute 18 cas : certaines fonctions de test sont paramétrées sur plusieurs entrées. Dix-huit cas ne signifie pas dix-huit fonctions distinctes. Les tests ne nécessitent ni clé ni requête extérieure.

Les contrôles couvrent réponses mal formées, réponse vide, timestamps non alignés, doublons, prix invalide, valeur infinie, jour courant, dernier snapshot valide, calendrier, calculs à la main, correction historique, erreurs HTTP et échec partiel. Le test dashboard vérifie l'état vide, l'absence d'exception et les filtres.

Le test d'upsert charge plusieurs fois trois dates, corrige un prix et vérifie qu'il reste trois lignes. Le test partiel commence avec des données valides, fait échouer Ethereum puis vérifie la conservation de son ancien prix et la mise à jour de Bitcoin. Ces assertions vérifient un résultat concret, pas seulement que la fonction ne plante pas.

AppTest de Streamlit exécute l'application sans ouvrir un navigateur. Cela ne remplace pas un contrôle visuel et ne teste pas tous les navigateurs. Des cas supplémentaires seraient utiles : plusieurs devises, rollback sur panne SQL, arrêt brutal et absence de clé. Aucun pourcentage de couverture n'a été mesuré.

18. GitHub, CI et protection des secrets

Git conserve l'historique local ; GitHub héberge les commits publiés. git status montre l'état du dossier ; git add prépare un commit ; git commit crée un jalon ; git push transmet le jalon. La branche main porte maintenant la V1. Le tag version-avancee-archive conserve le code précédent.

.gitignore exclut .env, les bases, les archives data, les logs, les caches et .venv. Un fichier déjà commité n'est pas retiré automatiquement par cette règle. Si un vrai secret a été publié, il faut d'abord le révoquer puis traiter l'historique ; simplement supprimer sa ligne ne suffit pas.

La clé est chargée depuis l'environnement, exclue du repr de Config et passée dans un en-tête. Le pipeline ne copie pas les messages d'erreur externes. Une variable d'environnement n'est pas un coffre chiffré : ces choix réduisent surtout l'exposition accidentelle dans un outil local.

GitHub Actions exécute installation, Ruff, formatage et pytest sur Windows/Linux avec Python 3.10/3.12. Le workflow ne collecte aucune donnée et n'a besoin d'aucune clé CoinGecko. Une CI verte prouve seulement que ces contrôles passent pour le commit exécuté dans ces environnements.

Le guide avancé de 57 pages est archivé et ne décrit plus cette V1. Pour préparer ton entretien, utiliser le présent guide et le README actuel. Il n'est pas nécessaire de présenter sur ton CV les fonctionnalités d'une ancienne version que tu ne maîtrises pas.

19. Limites à connaître et difficultés à expliquer

Les prix sont des snapshots CoinGecko, pas une clôture officielle. Les réponses peuvent manquer ou être révisées. Le volume est observé sur 24 h et ne doit pas être additionné comme si chaque point couvrait une période indépendante. Le projet ne garantit pas la qualité intrinsèque du fournisseur.

La collecte réelle n'a pas encore été validée avec une clé dans notre session. Le client est implémenté et testé par simulation, ce qui n'est pas la même preuve. La documentation fournisseur doit être revérifiée avant un premier usage réel ; aucun quota commercial permanent n'est garanti par le code.

L'outil est local : pas d'authentification, de haute disponibilité, de scheduler automatique, de cache, de migrations ou de gestion élaborée de concurrence. Le client n'attend pas automatiquement Retry-After. Les dépendances sont bornées mais pas verrouillées exactement. Les archives n'ont pas de politique automatique de rétention.

La V1 conserve les lignes valides antérieures si une nouvelle collecte ne les fournit plus. Elle ne conserve pas en SQL toutes les anciennes versions d'un prix. Elle peut laisser une exécution running après un arrêt brutal. Ces compromis doivent être expliqués, pas masqués.

Incident utile à raconter : dans l'ancienne interface, un module gardé en mémoire ne produisait pas la nouvelle colonne trend. Le diagnostic a comparé le serveur existant à un processus neuf. Le redémarrage et la surveillance du dossier src ont corrigé ce problème de chargement. La colonne trend n'existe plus dans la V1 ; cet incident illustre une difficulté passée, pas une fonctionnalité actuelle.

20. Plan d'apprentissage sur quatre semaines

Semaine 1 : lire config et client, reproduire un objet JSON simulé, puis transformer une série. Livrable personnel : expliquer les entrées et sorties et justifier la jointure par timestamp. Exercice : retirer une capitalisation et vérifier qu'elle ne glisse pas vers le mauvais prix.

Semaine 2 : comprendre la table SQLite et la clé composite, charger deux fois les mêmes dates, corriger un prix, simuler un échec. Livrable : montrer trois lignes conservées après deux chargements et la nouvelle valeur après correction. Expliquer la différence entre nouvelle ligne et ligne traitée.

Semaine 3 : refaire les quatre indicateurs à la main, modifier une période et suivre les colonnes jusqu'au graphique. Livrable : démonstration de la page, sans appeler l'API à chaque filtre. Exercice : introduire un jour absent et expliquer les valeurs indisponibles.

Semaine 4 : lancer les contrôles, écrire ou adapter un test simple, relire le README et préparer une présentation de cinq minutes. Livrable : une modification personnelle expliquée avec son test et une description CV proportionnée à ce que tu sais démontrer.

Environ 6 à 10 heures par semaine est une estimation de travail, pas une garantie. Si Python et SQL sont nouveaux, réduire la première étape à Bitcoin puis ajouter les autres après compréhension. Garder le périmètre stable : finir la chaîne de données avant d'ajouter un nouvel indicateur.

21. Valoriser le projet sur le CV selon ton avancement

Si tu commences seulement, écrire « Projet personnel en cours : apprentissage d'un pipeline API → pandas → SQLite et d'un dashboard Streamlit ». Tu peux mentionner l'objectif et les technologies étudiées, mais pas affirmer une maîtrise ni une collecte réelle terminée. Ne mets pas un projet en cours au même niveau qu'une réalisation professionnelle livrée.

Après avoir compris, exécuté et modifié la V1, titre conseillé : « Key Crypto - Pipeline ETL et dashboard d'analyse crypto ». Ligne de technologies : « Python · pandas · API REST · SQLite · Streamlit · Plotly ». NumPy, Matplotlib, corrélations et détection de motifs ne font pas partie de la V1 à valoriser.

Version courte prête à adapter : « Développement d'un pipeline Python de collecte et de nettoyage des données de trois cryptomonnaies ; stockage SQLite sans doublons et dashboard interactif de comparaison des performances. » Utiliser cette formulation lorsque tu peux défendre ses étapes. Si tu as surtout étudié le code généré, préférer « Réalisation avec assistance IA et appropriation d'un pipeline... ».

Version en trois puces, après appropriation : « Collecte API et nettoyage des données quotidiennes de Bitcoin, Ethereum et Solana avec pandas. » ; « Stockage SQLite avec upserts et tests des mises à jour sans doublons. » ; « Dashboard Streamlit/Plotly : variations, MM7, volatilité, comparaison base 100 et export CSV. » Ces phrases expriment des réalisations concrètes sans vocabulaire gonflé.

Les chiffres défendables sont trois actifs, 90 jours configurés, quatre indicateurs et 18 cas de test existants. Les 270 lignes sont synthétiques sur une base neuve, pas des données réelles collectées. Ne reprendre ni 50 000 observations, ni millions de lignes, ni gain de performance, ni taux de couverture sans mesure correspondante.

Ajouter un lien court vers le dépôt sous le titre. Le README permet au recruteur de retrouver l'installation, les tests et les limites. La qualité de la démonstration et ta capacité à expliquer les cas difficiles valent davantage qu'une longue liste de technologies.

22. Adapter le CV au poste visé

Pour un stage data analyst : mettre en avant la préparation des données et l'interprétation. Proposition : « Préparation de séries quotidiennes avec pandas et création d'un dashboard de comparaison en base 100, moyennes mobiles et volatilité ; gestion explicite des données manquantes. » Préparer un exemple où un trou empêche une moyenne valide.

Pour un stage data engineering ou Python : mettre en avant l'ingestion, la qualité et le stockage. Proposition : « Mise en œuvre d'un pipeline API REST → pandas → SQLite, avec validation des observations, upserts transactionnels et conservation des données après échec partiel. » Préparer une explication de la clé composite et du lot transactionnel.

Pour un profil généraliste : privilégier la chaîne complète. Proposition : « Réalisation d'une application locale Python reliant collecte API, stockage SQL et dashboard interactif, accompagnée de tests automatisés et d'une documentation d'installation. » Les détails de statistiques peuvent être réservés à l'entretien.

La CI est un élément de méthode complémentaire. Tu peux écrire « Vérifications automatisées avec pytest et GitHub Actions » si tu sais lire le workflow et expliquer qu'il ne collecte pas les prix. Ne pas écrire « déploiement automatique » : aucune application n'est déployée par cette CI.

Éviter : « expert des marchés crypto », « prédiction », « outil de trading », « infrastructure scalable », « production » et « haute disponibilité ». Le projet est une réalisation locale d'apprentissage. Une formulation précise peut être ambitieuse sans être trompeuse.

23. Présenter le projet en entretien et expliquer l'assistance IA

Pitch de 30 secondes, après appropriation : « J'ai travaillé sur un projet Python qui récupère des données CoinGecko, les nettoie et les conserve dans SQLite sans doublons. Une page Streamlit permet de comparer trois actifs et leurs indicateurs simples. J'ai utilisé une assistance IA, puis vérifié les traitements avec des tests et travaillé les limites sur les dates et les données manquantes. » Adapter au travail réellement effectué.

Présentation de deux minutes : expliquer d'abord le besoin ; dessiner le flux en cinq étapes ; choisir une difficulté, comme le double chargement ou un jour manquant ; montrer le test correspondant ; finir par une limite et l'amélioration suivante. Ne pas réciter toutes les fonctions.

Si l'on demande « Qu'as-tu fait toi-même ? », citer des actions vérifiables : lecture et explication d'une fonction, modification d'une règle, ajout d'un test, diagnostic d'un comportement, installation et démonstration. Ne pas prétendre avoir écrit sans aide du code fourni par un assistant. Tu peux valoriser ta capacité de vérification et d'appropriation.

Si tu ne maîtrises pas encore une partie, dire « Je comprends le rôle de la transaction mais je dois encore approfondir son comportement en cas de concurrence ». Une limite explicitée est plus défendable qu'une réponse improvisée. Le projet fournit un support concret pour apprendre.

Questions de relance à préparer : pourquoi SQLite, pourquoi ne pas interpoler, pourquoi 31 prix pour la volatilité, que fait un upsert, pourquoi une variable d'environnement, que prouvent les tests et ce qu'il faudrait ajouter avant un déploiement partagé. Les réponses suivantes donnent des repères, à reformuler avec tes mots.

24. Questions-réponses techniques

Pourquoi séparer le pipeline et le dashboard ?

Le pipeline modifie la base ; le dashboard consulte son état. Cela évite des appels réseau à chaque filtre, facilite les tests et permet de garder une dernière version consultable après une erreur de collecte.

Pourquoi SQLite plutôt qu'un CSV ?

SQLite fournit une clé primaire, des contraintes et des transactions. Un CSV est utile pour exporter mais ne gère pas lui-même un conflit d'unicité. Le stockage doit permettre de rejouer une collecte sans ajouter de doublons.

Qu'est-ce qu'un upsert ?

Une insertion avec mise à jour si la clé existe déjà. Dans le projet, le conflit porte sur actif, devise et date. Le test vérifie qu'une correction remplace le prix sans augmenter le nombre de lignes.

Pourquoi ne pas combler un prix absent ?

Le remplissage inventerait une valeur et pourrait créer un rendement nul artificiel, donc modifier la volatilité. La V1 garde le trou et rend indisponibles les calculs qui exigent ce prix.

Pourquoi une MM7 peut-elle exister au début de la période filtrée ?

Les calculs utilisent toute l'histoire disponible avant le filtre. Les jours antérieurs à la sélection peuvent compléter la fenêtre. La base 100, elle, est recalculée après le filtre car elle doit commencer à 100 sur la sélection.

Pourquoi 31 prix pour 30 jours de volatilité ?

Chaque rendement exige deux prix successifs. Une série de 31 prix fournit 30 rendements. L'écart-type est calculé sur ces rendements, pas sur les niveaux de prix.

Le projet prédit-il le marché ?

Non. Les indicateurs résument l'historique ; aucune stratégie, performance prédictive ou rentabilité n'est testée. Les graphiques permettent de comparer et de comprendre les observations.

Que se passe-t-il après une réponse HTTP 429 ?

La collecte de cet actif échoue sans reprise automatique. Le pipeline continue pour les autres et conserve les données anciennes. L'utilisateur attend puis relance manuellement. Les reprises avancées ont été volontairement reportées.

Les 18 tests prouvent-ils que tout fonctionne ?

Ils vérifient des comportements définis sur des entrées choisies. Ils ne remplacent ni une collecte authentifiée, ni un test de charge, ni tous les contrôles visuels. Aucune couverture exhaustive n'est revendiquée.

Comment évoluer vers un usage partagé ?

Commencer par mesurer le besoin : concurrence, volume, fréquence et sécurité. Ajouter ensuite une gestion d'accès, une planification surveillée, des erreurs structurées et éventuellement PostgreSQL. Ne pas complexifier la V1 sans besoin concret.

25. Exercices corrigés pour vérifier ton niveau

Exercice 1 : prix 100,105,110. Résultats : rendement initial indisponible, puis 5 % et environ 4,7619 %. Base 100 : 100,105,110. La performance totale est 10 %, pas la somme exacte des deux rendements.

Exercice 2 : sept prix de 100 à 106. La MM7 du dernier jour est 103 ; les six premières sont manquantes. Retirer le quatrième prix : la fenêtre reste incomplète et sa moyenne doit être manquante.

Exercice 3 : comparer deux actifs, A passant de 200 à 220 et B de 50 à 60. Leurs bases finales sont 110 et 120. B progresse davantage relativement même si son prix final reste plus faible. Les dates de début doivent être comparables.

Exercice 4 : prix constant pendant 31 jours. Après initialisation, les rendements valent zéro, la MM7 vaut ce prix et la volatilité vaut zéro. Les premiers résultats restent indisponibles tant que les fenêtres complètes n'existent pas.

Exercice 5 : trois dates chargées deux fois puis correction de la première. Résultat : trois lignes, prix corrigé et collected_at actualisé. Écrire une assertion sur le nombre de lignes et une autre sur le prix ; ne pas seulement vérifier que la commande termine.

Exercice 6 : simuler une réponse 429 avec Mock. Attendre un appel, une erreur et aucune boucle de reprise. Simuler un échec Ethereum après une première collecte valide : attendre partial, ancien prix Ethereum et nouveaux prix des autres actifs.

Exercice 7 : modifier toi-même la MM7 en MM5 dans une branche d'exercice. Mettre à jour le nom de colonne, le graphique, le test et la documentation. Ne publier cette modification que si le périmètre choisi la justifie ; le but est de montrer que tu sais suivre les dépendances.

26. Démonstration de cinq minutes et liste de contrôle

Minute 1 : présenter le besoin et préciser que la démo est synthétique. Minute 2 : montrer base 100 puis prix et MM7. Minute 3 : expliquer le calendrier et la volatilité. Minute 4 : montrer la clé SQL et le test d'upsert. Minute 5 : citer les limites et ta prochaine amélioration.

Avant l'entretien, exécuter la démo, ouvrir le serveur et lancer pytest. Préparer les fichiers client, transform, storage, indicators et un test. Garder une capture ou le PDF en cas de problème local. Ne pas dépendre d'un appel réseau réel pour expliquer le nettoyage.

Tu es prêt à présenter la V1 lorsque tu sais expliquer le flux sans lire, calculer les quatre indicateurs simples sur un exemple, retrouver l'upsert, justifier un trou, lire une assertion et modifier une règle avec son test. Aucun besoin de maîtriser une régression ou un RSI retiré du périmètre.

La prochaine étape raisonnable est de valider une collecte réelle avec ta clé, puis d'ajouter un petit test personnel. Plus de graphiques ne sont pas prioritaires. Conserver un périmètre modeste et le défendre précisément apporte une valeur claire sur un CV étudiant.

27. Glossaire et références

API REST : interface accessible par requêtes HTTP. Endpoint : adresse d'une ressource. JSON : format textuel de données structurées. Timestamp : valeur représentant un instant. UTC : référence temporelle commune. Snapshot : observation à un instant donné. ETL : extraire, transformer, charger.

DataFrame : tableau pandas. Index : clé des lignes. NaN : valeur numérique manquante en mémoire. NULL : absence de valeur SQL. Clé primaire : identifiant unique d'une ligne. Upsert : insertion ou mise à jour sur conflit. Transaction : groupe d'écritures validé ou annulé. Idempotence : répétition préservant l'état métier attendu.

Rendement : variation relative de prix. Moyenne mobile : moyenne d'une fenêtre glissante. Volatilité : dispersion des rendements selon une définition précisée. Base 100 : normalisation du prix par un prix initial. CI : contrôles automatiques après modification. Mock : objet simulé utilisé pour tester sans service externe.

Référence du comportement : code V1, README, pyproject.toml et tests reproduits en annexe. Documentation fournisseur déjà consultée pour le développement : https://www.coingecko.com/learn/download-bitcoin-historical-data et https://support.coingecko.com/hc/en-us/articles/4538879945625-API-key-can-I-store-the-key-in-my-headers . Les contrats et offres doivent être revérifiés avant une collecte réelle. Les numéros de ligne de l'annexe correspondent à cette édition.

Annexe A. Fiches des modules et lecture du code

Chaque source ci-dessous est reproduite depuis le dépôt local. Les numéros facilitent la discussion. Une ligne longue peut être poursuivie sur la ligne suivante sans nouveau numéro. Les commentaires et tests font partie du comportement documenté.

Méthode de lecture : identifier les entrées, suivre les transformations, repérer les effets de bord, puis associer une assertion de test à chaque garantie importante.

A1. pyproject.toml

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
001  [build-system]
002  requires = ["setuptools>=68"]
003  build-backend = "setuptools.build_meta"
004
005  [project]
006  name = "crypto-etl-local"
007  version = "1.0.0"
008  requires-python = ">=3.10"
009  dependencies = ["pandas>=2.2,<3", "requests>=2.32,<3", "streamlit>=1.49,<2", "plotly>=5.24,<7"]
010
011  [project.optional-dependencies]
012  dev = ["pytest>=8,<9", "ruff>=0.9,<1"]
013
014  [project.scripts]
015  crypto-etl = "crypto_etl.cli:main"
016
017  [tool.setuptools.packages.find]
018  where = ["src"]
019
020  [tool.pytest.ini_options]
021  testpaths = ["tests"]
022  pythonpath = ["src"]
023
024  [tool.ruff]
025  line-length = 100
026
```

A2. .env.example

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
# Définir ces variables dans le terminal ; .env n'est pas chargé automatiquement.
COINGECKO_API_KEY=
CRYPTO_CURRENCY=usd
CRYPTO_DAYS=90
CRYPTO_DATA_DIR=data

```

A3. .gitignore

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
.env
.venv/
__pycache__/
*.py[cod]
*.egg-info/
.pytest_cache/
.ruff_cache/
data/
*.db*
*.log
build/
dist/

```

A4. src/crypto_etl/config.py

Suivre Config, sa validation et load_config. Distinguer valeurs par défaut, variables réellement configurables et secret exclu de repr.

```
"""Configuration commune à la CLI et au dashboard."""

import os
from dataclasses import dataclass, field
from pathlib import Path


@dataclass(frozen=True)
class Config:
    api_key: str = field(default="", repr=False)
    currency: str = "usd"
    days: int = 90
    data_dir: Path = Path("data")
    coins: tuple[str, ...] = ("bitcoin", "ethereum", "solana")
    timeout: float = 30
    base_url: str = "https://api.coingecko.com/api/v3"

    def __post_init__(self):
        if not 1 <= self.days <= 365:
            raise ValueError("CRYPTO_DAYS doit être compris entre 1 et 365")
        if not self.currency.isalpha() or self.timeout <= 0:
            raise ValueError("Configuration invalide")

    def db_path(self, demo=False):
        return self.data_dir / ("demo.db" if demo else "market.db")


def load_config():
    return Config(
        api_key=os.getenv("COINGECKO_API_KEY", ""),
        currency=os.getenv("CRYPTO_CURRENCY", "usd").lower(),
        days=int(os.getenv("CRYPTO_DAYS", "90")),
        data_dir=Path(os.getenv("CRYPTO_DATA_DIR", "data")),
    )
```

A5. src/crypto_etl/client.py

Séparer décodage JSON et validation de structure. Vérifier la tentative unique et le timeout. Repérer les erreurs immédiatement refusées.

```
"""Un appel manuel par actif, timeout et messages sans secret."""

import requests


def validate_payload(payload):
    if not isinstance(payload, dict):
        raise TypeError("Réponse JSON non objet")
    for key in ("prices", "market_caps", "total_volumes"):
        values = payload.get(key)
        if not isinstance(values, list):
            raise TypeError(f"Série absente ou invalide : {key}")
        if any(not isinstance(p, (list, tuple)) or len(p) != 2 for p in values):
            raise ValueError(f"Paires invalides : {key}")
    return payload


class Client:
    def __init__(self, config, session=None):
        self.config = config
        self.session = session or requests.Session()

    def fetch(self, coin):
        c = self.config
        if not c.api_key:
            raise ValueError("COINGECKO_API_KEY requis pour une collecte réelle")
        try:
            response = self.session.get(
                f"{c.base_url}/coins/{coin}/market_chart",
                params={"vs_currency": c.currency, "days": c.days, "interval": "daily"},
                headers={"x-cg-demo-api-key": c.api_key},
                timeout=c.timeout,
            )
        except requests.RequestException:
            raise RuntimeError("Erreur réseau : relancer la collecte plus tard") from None
        if response.status_code != 200:
            raise ValueError(f"HTTP {response.status_code} : collecte refusée")
        try:

            return response.json()
        except ValueError:
            raise ValueError("JSON illisible") from None
```

A6. src/crypto_etl/transform.py

Suivre les clés d'alignement et le moment où la date devient quotidienne. Justifier chaque filtre et chaque keep='last'.

```
"""Alignement exact des timestamps puis dernier snapshot valide du jour UTC."""

import math

import pandas as pd

from .client import validate_payload
from .quality import quality_report


def transform(payload, coin, currency, days=90, now=None):
    validate_payload(payload)
    today = pd.Timestamp(now or pd.Timestamp.now(tz="UTC"))
    today = today.tz_localize("UTC") if today.tzinfo is None else today.tz_convert("UTC")
    today = today.normalize().tz_localize(None)
    start = today - pd.Timedelta(days=days)
    series = []
    duplicates = 0
    for key, name in (
        ("prices", "price"),
        ("market_caps", "market_cap"),
        ("total_volumes", "volume_24h"),
    ):
        part = pd.DataFrame(payload[key], columns=["source_timestamp", name])
        part["source_timestamp"] = pd.to_numeric(part["source_timestamp"], errors="coerce")
        part = part.dropna(subset=["source_timestamp"])
        duplicates += int(part.duplicated("source_timestamp").sum())
        part = part.drop_duplicates("source_timestamp", keep="last")
        series.append(part.set_index("source_timestamp"))
    frame = pd.concat(series, axis=1).reset_index()
    frame["date_utc"] = (
        pd.to_datetime(frame["source_timestamp"], unit="ms", utc=True, errors="coerce")
        .dt.tz_localize(None)
        .dt.normalize()
    )
    for column in ("price", "market_cap", "volume_24h"):
        frame[column] = pd.to_numeric(frame[column], errors="coerce")
        valid = frame[column].map(math.isfinite) & (frame[column] >= 0)

        frame.loc[~valid, column] = float("nan")
    invalid = frame["price"].isna() | (frame["price"] <= 0) | frame["date_utc"].isna()
    rejected = int(invalid.sum())
    excluded = int((frame["date_utc"] >= today).sum())
    frame = frame.loc[~invalid & frame["date_utc"].between(start, today, inclusive="left")]
    frame = frame.sort_values("source_timestamp").drop_duplicates("date_utc", keep="last")
    frame = frame.sort_values("date_utc").reset_index(drop=True)
    report = quality_report(
        frame, rejected, duplicates, excluded, start, today - pd.Timedelta(days=1)
    )
    frame["coin_id"] = coin
    frame["vs_currency"] = currency
    return frame, report
```

A7. src/crypto_etl/quality.py

Comparer calendrier attendu et dates acceptées. Expliquer pourquoi les compteurs ne s'additionnent pas forcément au nombre de points bruts.

```
"""Bilan sérialisable, sans interpolation."""

import pandas as pd


def quality_report(frame, rejected, duplicates, excluded, start, end):
    expected = pd.date_range(start, end, freq="D")
    observed = pd.DatetimeIndex(frame["date_utc"])
    prices = frame.set_index("date_utc")["price"].reindex(expected)
    returns = prices.pct_change(fill_method=None)
    return {
        "accepted": len(frame),
        "rejected": rejected,
        "duplicate_timestamps": duplicates,
        "excluded_current_or_future": excluded,
        "missing_days": expected.difference(observed).strftime("%Y-%m-%d").tolist(),
        "missing_market_cap": int(frame["market_cap"].isna().sum()),
        "missing_volume": int(frame["volume_24h"].isna().sum()),
        "large_changes": returns[returns.abs() > 0.5].index.strftime("%Y-%m-%d").tolist(),
    }
```

A8. src/crypto_etl/storage.py

Lire la clé primaire, les CHECK et l'upsert. Repérer les placeholders, le lot transactionnel et la fermeture explicite avec closing.

```
"""Transactions SQLite : une crypto validée est chargée atomiquement."""

import sqlite3
from contextlib import closing, contextmanager

import pandas as pd


@contextmanager
def connect(path):
    path.parent.mkdir(parents=True, exist_ok=True)
    with closing(sqlite3.connect(path)) as db, db:
        db.executescript("""
        CREATE TABLE IF NOT EXISTS daily_market_data (
            coin_id TEXT NOT NULL, vs_currency TEXT NOT NULL, date_utc TEXT NOT NULL,
            source_timestamp INTEGER NOT NULL, price REAL NOT NULL CHECK(price>0),
            market_cap REAL CHECK(market_cap>=0), volume_24h REAL CHECK(volume_24h>=0),
            collected_at TEXT NOT NULL,
            PRIMARY KEY (coin_id, vs_currency, date_utc));
        CREATE TABLE IF NOT EXISTS pipeline_runs (
            run_id TEXT PRIMARY KEY, started_at TEXT NOT NULL, ended_at TEXT,
            status TEXT NOT NULL, rows_processed INTEGER NOT NULL DEFAULT 0,
            details TEXT NOT NULL DEFAULT '{}');
        """)
        yield db


def upsert(db, frame, collected_at):
    rows = []
    for row in frame.to_dict("records"):
        rows.append(
            (
                row["coin_id"],
                row["vs_currency"],
                row["date_utc"].strftime("%Y-%m-%d"),
                int(row["source_timestamp"]),
                float(row["price"]),
                None if pd.isna(row["market_cap"]) else float(row["market_cap"]),

                None if pd.isna(row["volume_24h"]) else float(row["volume_24h"]),
                collected_at,
            )
        )
    with db:
        db.executemany(
            """
        INSERT INTO daily_market_data VALUES (?,?,?,?,?,?,?,?)
        ON CONFLICT(coin_id, vs_currency, date_utc) DO UPDATE SET
        source_timestamp=excluded.source_timestamp, price=excluded.price,
        market_cap=excluded.market_cap, volume_24h=excluded.volume_24h,
        collected_at=excluded.collected_at
        """,
            rows,
        )


def read_data(path, currency):
    with connect(path) as db:
```

```
        return pd.read_sql_query(
            "SELECT * FROM daily_market_data WHERE vs_currency=? ORDER BY date_utc",
            db,
            params=(currency,),
            parse_dates=["date_utc"],
        )


def latest_run(path):
    with connect(path) as db:
        db.row_factory = sqlite3.Row
        row = db.execute("SELECT * FROM pipeline_runs ORDER BY started_at DESC LIMIT 1").fetchone()
        return dict(row) if row else None
```

A9. src/crypto_etl/pipeline.py

Suivre le run_id du fichier brut à pipeline_runs. Montrer la limite de transaction par actif et le cas d'échec partiel.

```
"""Orchestration, archives brutes et suivi des échecs par crypto."""

import json
import logging
from datetime import datetime, timezone
from uuid import uuid4

from .client import Client
from .storage import connect, upsert
from .transform import transform

logger = logging.getLogger(__name__)


def utc_now():
    return datetime.now(timezone.utc).isoformat()


def run(config, fetch=None, demo=False):
    fetch = fetch or Client(config).fetch
    run_id, started = uuid4().hex, utc_now()
    raw_dir = config.data_dir / ("raw-demo" if demo else "raw") / run_id
    raw_dir.mkdir(parents=True, exist_ok=True)
    details, rows, successes = {}, 0, 0
    with connect(config.db_path(demo)) as db:
        with db:
            db.execute(
                "INSERT INTO pipeline_runs(run_id,started_at,status) VALUES (?,?,?)",
                (run_id, started, "running"),
            )
        for coin in config.coins:
            try:
                payload = fetch(coin)
                (raw_dir / f"{coin}.json").write_text(json.dumps(payload), encoding="utf-8")
                frame, report = transform(payload, coin, config.currency, config.days)
                if frame.empty:
                    raise ValueError("Aucune observation valide")
                upsert(db, frame, utc_now())

                details[coin] = {"status": "success", "quality": report}
                rows += len(frame)
                successes += 1
            except Exception as error:  # noqa: BLE001 — isoler chaque crypto dans le pipeline
                # Les exceptions externes peuvent contenir des secrets : ne pas les sérialiser.
                details[coin] = {"status": "failed", "error_type": type(error).__name__}
                logger.error("Collecte échouée pour %s (%s)", coin, type(error).__name__)
        status = (
            "success" if successes == len(config.coins) else "partial" if successes else
    "failed"
        )
        with db:
            db.execute(
                """UPDATE pipeline_runs SET ended_at=?, status=?, rows_processed=?,
                       details=? WHERE run_id=?""",
                (utc_now(), status, rows, json.dumps(details), run_id),
            )
    return {"run_id": run_id, "status": status, "rows": rows, "details": details}
```

A10. src/crypto_etl/indicators.py

Vérifier groupby, asfreq et ordre calcul/filtre. Expliquer pourquoi la base 100 dépend de la sélection et la MM7 utilise l'histoire antérieure.

```
"""Fenêtres calendaires complètes ; volatilité non annualisée en pourcentage."""

import pandas as pd


def indicators(frame):
    parts = []
    for (coin, currency), group in frame.groupby(["coin_id", "vs_currency"]):
        daily = group.set_index("date_utc").sort_index().asfreq("D")
        daily["coin_id"], daily["vs_currency"] = coin, currency
        price = daily["price"]
        daily["return_1d"] = price.pct_change(fill_method=None) * 100
        daily["ma_7"] = price.rolling(7, min_periods=7).mean()
        daily["volatility_30d"] = daily["return_1d"].rolling(30, min_periods=30).std(ddof=1)
        parts.append(daily.reset_index())
    return pd.concat(parts, ignore_index=True) if parts else frame.copy()


def select_period(frame, start, end):
    selected = frame.loc[frame["date_utc"].between(pd.Timestamp(start),
pd.Timestamp(end))].copy()
    if selected.empty:
        return selected
    groups = selected.groupby(["coin_id", "vs_currency"])["price"]
    selected["base_100"] = 100 * selected["price"] / groups.transform("first")
    return selected
```

A11. src/crypto_etl/demo.py

Identifier la formule synthétique et l'absence d'appel réseau. La date de fin change, même si les valeurs sont déterministes par position.

```
001  """Données déterministes synthétiques, sans requête HTTP."""
002
003  import math
004
005  import pandas as pd
006
007
008  def synthetic(coin, days=90):
009      base = {"bitcoin": 50000, "ethereum": 2500, "solana": 100}[coin]
010      dates = pd.date_range(
011          end=pd.Timestamp.now(tz="UTC").normalize() - pd.Timedelta(days=1), periods=days,
     freq="D"
012      )
013      payload = {"prices": [], "market_caps": [], "total_volumes": []}
014      for i, date in enumerate(dates):
015          timestamp = int(date.timestamp() * 1000)
016          price = base * (1 + i / 1000 + 0.08 * math.sin(i / 12 + base % 7))
017          for key, value in (
018              ("prices", price),
019              ("market_caps", price * 1e7),
020              ("total_volumes", price * 1e5 * (1 + 0.2 * math.cos(i))),
021          ):
022              payload[key].append([timestamp, value])
023      return payload
```

A12. src/crypto_etl/cli.py

Relier la commande à la base choisie et au code de sortie. L'absence de clé est vérifiée avant une collecte réelle.

```
"""Commandes manuelles utilisables également par un planificateur local."""

import argparse
import json
import logging

from .config import load_config
from .demo import synthetic
from .pipeline import run


def main():
    parser = argparse.ArgumentParser(description="ETL crypto local")
    parser.add_argument("command", choices=["update", "demo"])
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    config = load_config()
    if args.command == "update" and not config.api_key:
        parser.error("Définir COINGECKO_API_KEY ou utiliser la commande demo")
    result = run(
        config,
        fetch=(lambda coin: synthetic(coin, config.days)) if args.command == "demo" else None,
        demo=args.command == "demo",
    )
    print(json.dumps(result, indent=2, ensure_ascii=False))
    return 0 if result["status"] == "success" else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

A13. dashboard/app.py

Suivre les états vides, la devise, la sélection et les graphiques. Les écritures de marché appartiennent au pipeline, pas aux filtres.

```
"""Une page d'analyse ; les filtres consultent seulement SQLite."""

import pandas as pd
import plotly.express as px
import streamlit as st

from crypto_etl.config import load_config
from crypto_etl.indicators import indicators, select_period
from crypto_etl.storage import latest_run, read_data

st.set_page_config(page_title="Key Crypto - V1", page_icon=" ", layout="wide")
st.title("Key Crypto - V1")
st.caption("Bitcoin, Ethereum et Solana : collecte, stockage et analyse quotidienne.")
config = load_config()
demo = st.sidebar.toggle("Démonstration synthétique", value=True)
if demo:
    st.warning("DÉMONSTRATION : données synthétiques, sans valeur de marché réelle.")
path = config.db_path(demo)
if not path.exists():
    st.info("Initialisez les données depuis le dossier du projet :")
    st.code("crypto-etl demo" if demo else "crypto-etl update")
    st.stop()
data = read_data(path, config.currency)
if data.empty:
    st.info("Base vide pour cette devise. Lancez crypto-etl demo ou crypto-etl update.")
    st.stop()
run = latest_run(path)
if run:
    st.caption(f"Dernière collecte : {run['ended_at']} · statut : {run['status']}")
    if run["status"] != "success":
        st.warning("Collecte incomplète : les anciennes données valides sont conservées.")
today = pd.Timestamp.now(tz="UTC").normalize().tz_localize(None)
if (data.groupby("coin_id").date_utc.max() < today - pd.Timedelta(days=1)).any():
    st.warning("Certaines données sont anciennes : relancez la collecte manuellement.")
coins = st.sidebar.multiselect("Cryptomonnaies", config.coins, default=list(config.coins))
dates = st.sidebar.date_input(
    "Période",
    value=(data.date_utc.min().date(), data.date_utc.max().date()),

    min_value=data.date_utc.min().date(),
    max_value=data.date_utc.max().date(),
)
if not coins or len(dates) != 2:
    st.info("Sélectionnez des cryptomonnaies et une période complète.")
    st.stop()
selected = select_period(indicators(data[data.coin_id.isin(coins)]), *dates)
observed = selected.dropna(subset=["price"])
if observed.empty:
    st.info("Aucun prix sur cette période.")
    st.stop()
if selected.price.isna().any():
    st.warning("Des jours sont manquants : les prix ne sont pas interpolés.")
st.caption(f"Devise : {config.currency.upper()} · dates UTC · collecte : {config.days} jours")
st.subheader("Dernières observations de la sélection")
summary = observed.groupby("coin_id").tail(1)
st.dataframe(
    summary[["coin_id", "date_utc", "price", "return_1d", "volatility_30d"]],
    hide_index=True,
```

```
    width="stretch",
)
st.subheader("Comparaison des performances")
st.plotly_chart(
    px.line(
        selected,
        x="date_utc",
        y="base_100",
        color="coin_id",
        title="Base 100 au premier prix disponible de la sélection",
    ),
    width="stretch",
)
coin = st.selectbox("Actif à analyser", coins)
subset = selected[selected.coin_id == coin]
st.plotly_chart(
    px.line(
        subset, x="date_utc", y=["price", "ma_7"], title="Prix observé et moyenne mobile 7 jours"
    ),
    width="stretch",
)
st.plotly_chart(
    px.line(
        subset, x="date_utc", y="volatility_30d", title="Volatilité 30 jours - non annualisée (%)"
    ),
    width="stretch",
)
st.caption(
    "La moyenne exige 7 prix consécutifs ; la volatilité exige 31 prix pour 30 rendements. "
    "Les premiers résultats indisponibles restent vides. Les indicateurs décrivent le passé."
)
st.download_button(
    "Exporter la sélection CSV",
    selected.to_csv(index=False).encode("utf-8"),
    file_name="crypto-demo.csv" if demo else "crypto-coingecko.csv",
    mime="text/csv",
)
st.caption(
    "Prix observés, sans clôture officielle. "
    "Source des collectes réelles : [CoinGecko](https://www.coingecko.com/)."
)
```

A14. tests/test_core.py

Identifier les risques métier couverts et les assertions vérifiant un résultat, plutôt que seulement une absence d'exception.

```
"""Tests des risques métier, entièrement hors réseau."""

from unittest.mock import Mock

import pandas as pd
import pytest
import requests

from crypto_etl.client import Client, validate_payload
from crypto_etl.config import Config
from crypto_etl.indicators import indicators, select_period
from crypto_etl.pipeline import run
from crypto_etl.storage import connect, read_data, upsert
from crypto_etl.transform import transform


def payload(prices=(100, 105, 110), start="2024-01-01"):
    dates = pd.date_range(start, periods=len(prices), tz="UTC")
    pairs = [[int(d.timestamp() * 1000), p] for d, p in zip(dates, prices)]
    return {
        "prices": pairs,
        "market_caps": [[t, 10] for t, _ in pairs],
        "total_volumes": [[t, 20] for t, _ in pairs],
    }


@pytest.mark.parametrize("value", [None, [], {}, {"prices": [1]}])
def test_bad_payload(value):
    with pytest.raises((ValueError, TypeError)):
        validate_payload(value)


def test_empty():
    frame, quality = transform(payload(()), "bitcoin", "usd", 3, "2024-01-04")
    assert frame.empty and len(quality["missing_days"]) == 3


def test_alignment_duplicates_invalid_and_today():

    raw = payload((100, 105, -2, 120))
    raw["prices"] = raw["prices"][::-1] + [raw["prices"][0]]
    raw["market_caps"] = raw["market_caps"][1:]
    raw["total_volumes"][1][1] = float("inf")
    frame, quality = transform(raw, "bitcoin", "usd", 4, "2024-01-04")
    assert frame.price.tolist() == [100, 105]
    assert pd.isna(frame.iloc[0].market_cap)
    assert pd.isna(frame.iloc[1].volume_24h)
    assert quality["rejected"] == 1
    assert quality["duplicate_timestamps"] == 1
    assert quality["excluded_current_or_future"] == 1
    assert "2024-01-03" in quality["missing_days"]


def test_alignment_is_not_positional():
    raw = payload()
    raw["market_caps"] = [[raw["prices"][2][0], 30], [raw["prices"][0][0], 10]]
    frame, _ = transform(raw, "bitcoin", "usd", 3, "2024-01-04")
    assert frame.market_cap.iloc[0] == 10
```

```
    assert pd.isna(frame.market_cap.iloc[1])
    assert frame.market_cap.iloc[2] == 30


def test_daily_last_valid():
    raw = payload((100,))
    raw["prices"].append([raw["prices"][0][0] + 3600000, 110])
    frame, _ = transform(raw, "bitcoin", "usd", 1, "2024-01-02")
    assert frame.price.tolist() == [110]


def test_indicators_and_exact_calendar():
    frame, _ = transform(payload(tuple(range(100, 140))), "bitcoin", "usd", 40, "2024-02-10")
    result = indicators(frame)
    assert result.return_1d.iloc[1] == pytest.approx(1)
    assert result.ma_7.iloc[6] == 103
    assert result.volatility_30d.iloc[30] ==
pytest.approx(result.return_1d.iloc[1:31].std(ddof=1))
    assert pd.isna(result.volatility_30d.iloc[29])

    selection = select_period(result, "2024-01-05", "2024-01-10")
    assert selection.base_100.iloc[0] == 100
    assert selection.ma_7.iloc[-1] == 106
    missing = indicators(frame.drop(index=3))
    assert pd.isna(missing.return_1d.iloc[4])
    assert pd.isna(missing.ma_7.iloc[6])


def test_upsert_and_revision(tmp_path):
    path = tmp_path / "test.db"
    frame, _ = transform(payload(), "bitcoin", "usd", 3, "2024-01-04")
    with connect(path) as db:
        upsert(db, frame, "first")
        upsert(db, frame, "second")
        frame.loc[0, "price"] = 200
        upsert(db, frame, "third")
    stored = read_data(path, "usd")
    assert len(stored) == 3 and stored.price.iloc[0] == 200


def test_client_success():
    session = Mock()
    session.get.return_value = Mock(status_code=200)
    session.get.return_value.json.return_value = payload()
    assert Client(Config(api_key="test-only"), session).fetch("bitcoin") == payload()
    assert session.get.call_args.kwargs["timeout"] == 30
    assert session.get.call_args.kwargs["params"]["days"] == 90


@pytest.mark.parametrize("status", [400, 401, 429, 500])
def test_http_error_without_retry(status):
    session = Mock()
    session.get.return_value = Mock(status_code=status)
    with pytest.raises(ValueError, match=str(status)):
        Client(Config(api_key="test-only"), session).fetch("bitcoin")
    assert session.get.call_count == 1


def test_network_and_invalid_json():
    session = Mock()
    session.get.side_effect = requests.ConnectionError("secret")
    with pytest.raises(RuntimeError, match="Erreur réseau"):
        Client(Config(api_key="test-only"), session).fetch("bitcoin")
    session.get.side_effect = None
    session.get.return_value = Mock(status_code=200)
```

```
    session.get.return_value.json.side_effect = ValueError("secret")
    with pytest.raises(ValueError, match="JSON illisible"):
        Client(Config(api_key="test-only"), session).fetch("bitcoin")


def test_partial_pipeline_preserves_data(tmp_path):
    config = Config(data_dir=tmp_path)
    yesterday = (pd.Timestamp.now(tz="UTC").normalize() -
pd.Timedelta(days=1)).strftime("%Y-%m-%d")
    first = run(config, lambda _: payload((100,), yesterday))
    assert first["status"] == "success"

    def fetch(coin):
        if coin == "ethereum":
            raise RuntimeError("secret")
        return payload((200,), yesterday)

    second = run(config, fetch)
    assert second["status"] == "partial"
    data = read_data(config.db_path(), "usd").set_index("coin_id")
    assert data.loc["ethereum", "price"] == 100
    assert data.loc["bitcoin", "price"] == 200
    assert "secret" not in str(second)
    assert len(list((tmp_path / "raw").glob("*/*.json"))) == 5
```

A15. tests/test_dashboard.py

Comprendre AppTest, les données temporaires et les filtres. Ce test ne remplace pas un contrôle visuel dans le navigateur.

```
001  """Vérifier l'état vide et les filtres via le moteur Streamlit, sans serveur réseau."""
002
003  from pathlib import Path
004
005  from streamlit.testing.v1 import AppTest
006
007  from crypto_etl.config import Config
008  from crypto_etl.demo import synthetic
009  from crypto_etl.pipeline import run
010
011  APP = Path(__file__).resolve().parents[1] / "dashboard" / "app.py"
012
013
014  def test_dashboard_empty_and_demo(tmp_path, monkeypatch):
015      monkeypatch.setenv("CRYPTO_DATA_DIR", str(tmp_path))
016      app = AppTest.from_file(str(APP)).run()
017      assert not app.exception
018      assert "Initialisez" in app.info[0].value
019      config = Config(data_dir=tmp_path)
020      assert run(config, synthetic, demo=True)["status"] == "success"
021      app.run()
022      assert not app.exception
023      assert len(app.tabs) == 0
024      assert len(app.dataframe) == 1
025      app.sidebar.multiselect[0].set_value(["bitcoin"]).run()
026      assert not app.exception
027      assert len(app.dataframe[0].value) == 1
028      app.sidebar.multiselect[0].set_value([]).run()
029      assert not app.exception
030      assert "Sélectionnez" in app.info[0].value
```

A16. .github/workflows/ci.yml

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
name: Qualité Python
on: [push, pull_request]
jobs:
  checks:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        python: ['3.10', '3.12']
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: pip
      - run: python -m pip install -e ".[dev]"
      - run: ruff check .
      - run: ruff format --check .
      - run: pytest
```

A17. .streamlit/config.toml

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
[server]
# Surveiller aussi les modules du paquet installé en mode éditable.
folderWatchList = ["src"]
runOnSave = true

[browser]
gatherUsageStats = false
```

Annexe B. Commandes de révision

```
# Depuis le dossier du projet, PowerShell
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m ruff check .
.venv\Scripts\python.exe -m ruff format --check .
.venv\Scripts\python.exe -m crypto_etl.cli demo
.venv\Scripts\python.exe -m streamlit run dashboard/app.py

# Inspection Git
git status --short
git log --oneline
git remote -v
git ls-files

# Linux : remplacer les exécutables par .venv/bin/python
```

Ces commandes de vérification sont sans collecte réelle. La commande update nécessite une clé chargée dans l'environnement ; ne jamais la coller dans un document partagé ou un commit.